

The /v1/extract Internals

How Python Code and the LLM Together Turn a PDF Into JSON

Function-by-function, layer-by-layer trace of gateway/main.py - what is deterministic Python code, what is a hand-written prompt rule, and what is left to the model's own judgment. Every function name and line number below is read from the live source on 2026-08-21.

gateway/main.py - Qwen3.6-35B-A3B (vLLM) - PaddleOCR - all-MiniLM-L6-v2 embeddings

0. Two Different Kinds of Work, In One Pipeline

Every /v1/extract call is really two collaborating systems, not one:

- Deterministic Python code - file parsing, OCR dispatch, image rendering, prompt assembly, JSON parsing, and (this is the part people usually miss) almost all of the actual 'find this value' logic that happens AFTER the model responds.
- The LLM (Qwen3.6-35B-A3B) - reading the document (text and/or images), deciding what JSON field names to use for THIS specific document, and writing out the values.

The single most important fact to understand before anything else: the model is NOT filling in a fixed template. Unless the caller supplies `output_schema`, there is no predefined set of JSON keys anywhere in this system. The model decides the key names itself, guided only by natural-language rules in the prompt. Every piece of Python code downstream of the LLM call has to cope with that - which is why 'finding a value' after extraction means searching across lists of known name variants, never just `data['amount']`.

The trace, top to bottom

- 1. File to text/images - Python (`extract_pages`, `extract_page_images`, `ocr_image`)
- 2. Prompt assembly - Python, calling a hand-authored rules block (`EXTRACTION_RULES`)
- 3. Reading + deciding fields + extracting values - the LLM
- 4. Parsing the model's raw text back into a Python object - Python (`_try_json`)
- 5. Self-consistency merge (if requested) - Python (`_merge_by_vote`)
- 6. Finding values under whatever keys the model chose - Python (`_first`, `_find_line_items`, ...)
- 7. Deterministic math/shape validation - Python (`validate_extraction`, 14 checks)
- 8. Fixing what validation flagged - the LLM again (`_self_correct`), one pass
- 9. Removing accidental duplicate values - Python (`_dedupe_duplicate_values`)
- 10. Optional field-name normalization - Python + a separate embedding model, NOT the LLM (`canonicalize_fields`)

1. Turning the File Into Text and Images (Python)

extract_pages()

[main.py:475]

Dispatches purely on file extension. For a PDF, PyMuPDF (fitz) reads each page's native text layer first - this is instant and perfectly accurate for a digitally-generated PDF, and involves no model of any kind.

The interesting decision is what happens when native text is thin:

```
if len(text.strip()) < 300:
    pix = page.get_pixmap(matrix=fitz.Matrix(2, 2))
    ocr_text = ocr_image(buf.getvalue(), f"{filename} p{i+1}")
    if len(ocr_text.strip()) > len(text.strip()):
        text = ocr_text
```

A page with under 300 characters of native text is rendered to an image and sent through OCR; whichever version - native or OCR - is LONGER wins, per page, not per document. This is deliberately conservative: a page can have a small amount of genuine native text (a routing stamp) while its real content is a table rendered only as an image the text layer can't see. An earlier, lower threshold let such pages through untouched and silently dropped the table entirely.

*Other file types: images go straight to OCR; docx is parsed with python-docx (paragraphs + tables joined with '|');
xlsx/xls with pandas (each sheet -> CSV, one 'page' per sheet); txt/csv is split into ~10,000-character pseudo-pages so
later chunking still has page-like boundaries.*

ocr_image()

[main.py:461]

A blocking HTTP call to a separate PaddleOCR microservice (its own process, its own virtualenv, port 7780). This is plain, deterministic OCR - a character-recognition model, not the LLM - and it fails soft: any error returns an empty string rather than raising, so one bad page never kills the whole document.

extract_page_images()

[main.py:542]

Independently of text extraction, every page is ALSO rendered as a PNG - this is the raw material for the LLM's own vision encoder in the next stage. Pages that look like dense tables (many short lines, exactly where small-print decimals get misread) render at 3x zoom; everything else at 2x. This is still plain Python image rendering - no model runs here either.

2. Assembling the Prompt (Python code calling a hand-written rules block)

Before anything reaches the model, Python decides which of two message-building paths to use:

```
use_vision = bool(images) and len(images) <= MAX_VISION_PAGES and len(doc_text) <= SINGLE_SHOT_CHARS
# MAX_VISION_PAGES = 10 pages, SINGLE_SHOT_CHARS = 40,000 characters
```

Vision-hybrid message - `_vision_doc_messages()`

[main.py:898]

Builds one chat message containing BOTH the OCR/native text of the document AND the actual page images, in the same content array (the same shape OpenAI's vision API uses: a list of text and image_url parts). The model's own vision encoder - a 27-layer transformer built into its weights, not a separate call - processes the images directly alongside the text.

Text-only message - `answer_document()`

[main.py:948]

For documents that don't qualify for vision (too many pages, or too much text), Python instead builds a text-only prompt: one call for documents under 40,000 characters, or a map-reduce sequence (chunk, answer each chunk, reduce) for larger ones - covered in full in the companion 'Vision & OCR Pipeline' document; this document focuses on the extraction-and-JSON side.

The instruction set - `EXTRACTION_RULES`

[main.py:612-802]

This is a single, hand-written, ~190-line block of natural-language rules appended to the user's question whenever the task wants structured output. It is NOT a schema - it's prose the model reads and follows. A few of the rules verbatim, because they directly explain behavior seen in the output:

On how keys are chosen (there is no fixed schema)

```
"Vendor/seller AND buyer, when both appear, must be two separate nested
objects (e.g. \"vendor\": {...}, \"bill_to\": {...}) -- never merge them
into one party. Beyond that, the overall shape stays fully dynamic: pick
whatever top-level fields/groups fit what THIS document actually
contains -- do not force every document into one fixed section layout..."
```

On what counts as a real line item

```
"Hard rule: if item/description, quantity, unit price, AND amount are
ALL null/empty for a candidate row, NEVER create that line_item -- a row
with nothing in it is not a row... Do not create a line item whose
amount equals the invoice subtotal -- that's the subtotal itself leaking
into the table as a fake row, not a real purchased item..."
```

On self-checking before answering

```
"Before writing the JSON, state on one line how many distinct rows/
records appear in the text provided (e.g. 'Row count: 7'), then make
sure the JSON contains exactly that many entries..."
```

On output purity

```
"The ONLY non-JSON content allowed is the single 'Row count: N' line
above. No other explanation, preamble, summary, or markdown code fences
(```) -- the response must be that one line followed immediately by the
raw JSON object."
```

Other rules in the same block (not quoted in full here) cover: zero data loss / never truncate rows, multi-line description joining, multiple-address handling per entity, tax-rate inference only from printed evidence, currency inference from symbols plus document context, primary invoice-number/date disambiguation from PO/job/routing numbers, ISO date normalization, and OCR character-confusion awareness (0/O, 1/I, 5/S, 8/B).

3. What Happens Inside the LLM Call

The call itself is a plain, deterministic invocation of the model - not agentic, no tools, no back-and-forth:

```
body = {"model": MODEL, "messages": messages, "max_tokens": max_tokens,
        "temperature": 0.0, "seed": 42,
        "chat_template_kwargs": {"enable_thinking": False}}
if response_format:
    body["response_format"] = response_format    # vLLM guided decoding
```

Temperature 0 and a fixed seed (42) make every extraction call-to-call reproducible for the SAME input - this is deliberate, and different from the free-form /chat endpoint, which never pins these.

What the model is actually deciding, unassisted by any Python logic

- Which top-level JSON keys to use for this specific document's fields (vendor vs seller vs issuer; bill_to vs buyer vs customer; a nested object vs a flat field) - guided only by the prose rules above, not a schema, unless the caller passed output_schema.
- Which rows in the source table are genuine line items vs. notes/stamps/references that must go to a 'metadata' field instead.
- How to join a wrapped multi-line description into one field.
- Which of several similar-looking numbers on the page is the actual invoice number vs. a PO/job/routing number.
- How to resolve an OCR-ambiguous character by cross-referencing the page image it was also given (vision-hybrid path only).
- The values themselves - every number, date, and string in the output.

Optional: output_schema and guided decoding

If the caller supplies output_schema (a JSON Schema string), it is passed to vLLM as response_format. This is NOT Python post-processing - it constrains the model's TOKEN-LEVEL generation inside vLLM itself (guided decoding), so the model is structurally incapable of emitting a field the schema doesn't allow, or malformed JSON. Without it, the model's output shape is entirely its own choice, constrained only by the prose rules above.

4. Turning the Model's Text Back Into a Python Object

`_try_json()`

[main.py:1932]

The model's response is plain text (a 'Row count: N' line plus JSON), not a structured object - this function recovers the JSON from it in three escalating attempts:

```
# 1. try the whole response as JSON directly
try: return json.loads(text)
except: pass
# 2. look for a fenced ```json ... ``` block (most reliable if present)
fence = re.search(r"```(?:json)?\s*(\{.*\}|\[.*\])\s*```", text, re.DOTALL)
# 3. greedy fallback: find the first {...} or [...] anywhere in the text
for pattern in (r"\{.*\}", r"\[.*\]"):
    m = re.search(pattern, text, re.DOTALL)
```

If all three fail, the field is treated as unparseable - this is exactly what the `not_json` validation check (Section 6) catches, which is what triggers the automatic self-correction pass.

5. Self-Consistency: Merging N Independent Extractions

When the caller sets consistency > 1, the ENTIRE extraction above (prompt -> LLM call -> parse) runs N times independently and in parallel, then the N JSON objects are merged by `_merge_by_vote()` (main.py:2026) - a recursive, structure-aware majority vote, not a text diff:

- dict: recurse into each key independently, unioning key names across all samples (a key present in only some samples is still considered).
- list of dicts (e.g. `line_items`): rows are matched across samples by IDENTITY, not position - `_row_identity()` (main.py:1955) uses the item code/SKU if present, else the first 60 characters of the description, lowercased. A row only survives if a STRICT MAJORITY of the samples that produced a list also produced that same row - with consistency=2 this means BOTH samples must agree, specifically to stop one sample's hallucinated extra row surviving on a tie.
- scalar leaf (a string/number/bool): plain majority vote (Python's Counter); a tie keeps whichever sample's value was seen first.

This step cannot fix an error every sample makes identically - a genuinely unreadable character stays wrong through every consistency pass. Catching THAT class of error is the validator's job (Section 6), not this one.

6. Finding Values Under Whatever Keys the Model Chose

This is the section that answers the core question of 'how does the code know which key holds which value' - and the honest answer is: it doesn't know in advance, because the model didn't commit to a fixed schema. Every downstream function that needs a specific concept (an amount, a quantity, a vendor name) has to search across a small list of the names the model is KNOWN to commonly use for that concept, defined once as plain Python tuples:

```
_QTY_KEYS      = ("quantity", "qty")
_PRICE_KEYS    = ("unit_price", "price", "rate")
_AMOUNT_KEYS   = ("amount", "total", "line_total", "total_amount")
_DESC_KEYS     = ("description", "desc", "item_description")
_ITEM_KEYS     = ("item_number", "item_code", "item", "sku", "part_number")
_TOTAL_KEYS    = ("total", "balance_due", "grand_total")
_TAX_KEYS      = ("sales_tax", "tax", "tax_total", "tax_amount", "vat", "gst")
_COST_COMPONENT_KEYS = ("labor", "labor_total", "parts", "parts_total",
                        "sublet", "shop_supplies", "materials",
                        "misc_charges", "hazmat_fee", "environmental_fee")
```

`_first(d, keys)`

[main.py:1810]

The base primitive everything else is built on - walk a tuple of candidate key names in priority order, and return the first one present in the dict with a non-null value. This is the direct answer to 'how does a value get assigned to a concept': not by a fixed key name, but by checking an ordered list of known aliases and taking the first hit.

```
def _first(d, keys):
    for k in keys:
        if k in d and d[k] is not None:
            return d[k]
    return None
```

`_find_line_items(data)`

[main.py:1826]

The line-items array itself could be under 'line_items', 'items', 'products', or anything else the model chose - so this scans every top-level value in the output looking for a LIST OF DICTS whose first entry contains at least one recognizable item-shaped key (an amount, price, quantity, or cost-component key). The first list that matches that shape is treated as the line-items table.

`_row_amount(it)`

[main.py:1836]

A row's effective amount is either its own amount/total field (via `_first` over `_AMOUNT_KEYS`), or - if that's absent - the SUM of any cost-component fields present (labor + parts + sublet + ...). This exists because repair/service invoices commonly break a charge into parts instead of stating one total per row.

`_find_totals(data)`

[main.py:1847]

Totals (subtotal/tax/total) aren't always nested - some documents state them at the top level of the JSON, others under a nested 'totals'-shaped object. This function checks the top level first, then falls back to searching one level of nested dicts for the same key shape.

Presence checks for narrative fields (vendor, buyer, dates)

For fields that are prose rather than numbers, the code can't rely on numeric matching - instead it combines a small set of expected KEY name hints with a regex search of the SOURCE TEXT for the corresponding printed label, so it can tell 'the source clearly has a Bill To section, but the output has no matching field' apart from 'this document simply never had one':

Constant	What it matches
_VENDOR_KEY_HINTS	vendor, supplier, seller, issuer, from_company, merchant
_BUYER_KEY_HINTS	bill_to, buyer, customer, sold_to, billing, bill-to
_BUYER_LABEL_RE	regex for 'Bill To:' / 'Sold To:' / 'Customer:' printed in the source text
_DATE_LABEL_RE	regex for 'Invoice Date:' printed in the source text
_CURRENCY_SYMBOL_RE	regex for \$ / EUR / GBP / Rs. / JPY appearing in any amount string

7. Deterministic Validation - validate_extraction()

main.py:2144 -- pure Python, zero model calls

Using the key-finding primitives from Section 6, 14 independent checks run against the parsed JSON. None of them ask the model anything - they are arithmetic and shape checks against whatever values were actually found:

Check	What it computes
empty_line_item	A row with no description, quantity, price, or amount at all
wrong_amount	quantity * unit_price does not match the stated amount (with a subscription-billing exception where amount legitimately equals unit_price alone)
wrong_column	A numeric field holds text, or vice versa - likely a column shift
duplicate_row	Identical item/description/amount repeated - a common map-reduce artifact
total_mismatch	Line items don't sum to the stated subtotal, or subtotal+tax != total
subtotal_as_line_item	A row's amount equals the subtotal - likely the subtotal leaking in as a fake line (exempt for genuine single-line invoices)
vendor_missing / buyer_missing	Party detected via label regex in source text, absent from output
invoice_date_missing / due_date_missing	Date label detected in source, absent from output
invoice_number_missing	No invoice number field found via _first over invoice-number-shaped keys
currency_missing	Soft check - no currency symbol or field found anywhere
missing_row	Heuristic - a code/value pattern in the source text with no matching output row

Example - the actual reconciliation math behind total_mismatch:

```
line_sum = sum(_row_amount(it) or 0 for it in items)
fee_extra = sum(_num(totals.get(k)) or 0 for k in _EXTRA_FEE_KEYS if k in totals)
if items and subtotal is not None:
    candidates = (line_sum, line_sum + fee_extra)
    if not any(abs(c - subtotal) <= max(0.05, subtotal * 0.01) for c in candidates):
        issues.append({"type": "total_mismatch", "severity": "hard", ...})
```

Each issue is tagged with a severity. Only issues marked 'hard' trigger the self-correction pass in the next section - the soft/heuristic ones (currency_missing, missing_row) are reported but not auto-corrected.

8. Self-Correction - the LLM Gets a Second, Guided Attempt

`_self_correct()`

[main.py:2331]

If Section 7 found any hard issue, Python makes exactly ONE more LLM call: the model is given the original document (plus the original page images, if the vision-hybrid path was used the first time) AND the precise, itemized list of what `validate_extraction()` found wrong - not a vague 'please double check', but the literal issue descriptions ('row 3: 4 x 12.50 = 50.00, but stated amount is 45.00'). The corrected JSON is parsed the same way (Section 4) and run BACK through `validate_extraction()` so the final response always reflects the post-correction state.

This is the loop visible in the pipeline diagram in the companion Vision & OCR document: `validate` -> (if hard issues) `self_correct` -> `re-validate` -> `done`. It runs at most once per file - there is no `retry-until-clean` loop here, by design, to bound how many LLM calls one extraction can cost.

9. Removing Accidental Duplicate Values - Python, No Model

`_dedupe_duplicate_values()`

[main.py:1970]

Self-consistency voting (Section 5) can leave the same real-world fact under several plausible-sounding field names at different nesting depths - e.g. a top-level `account_number` AND an identical value under `metadata.account_no`, or `fees / fees_policy / fees_section` all holding the same paragraph verbatim. This function recurses through nested dicts and drops any string value (over 5 characters) that exactly duplicates one already seen earlier in the object's own key order - whichever occurrence came first wins. It deliberately does NOT touch `line_items` or any other list: two different rows that happen to share a value (e.g. two items at the same price) are never treated as duplicates.

10. Canonicalization - a Second, Smaller Model, Not the LLM

canonicalize_fields()

[main.py:2099]

Only runs if the caller explicitly sets canonicalize=true. Its entire job is fixing the fact that, with no output_schema, the SAME document processed twice can come back with 'vendor' one time and 'vendor_info' the next. Rather than a fixed lookup table of synonyms, it uses the embedding microservice already running for RAG (all-MiniLM-L6-v2) to measure semantic similarity between each top-level key the model chose and a small, hand-written set of canonical concepts:

```
CANONICAL_FIELDS = {
    "vendor": "the selling company, supplier, or vendor's name, address, phone, and email",
    "bill_to": "the billing customer's name and address",
    "ship_to": "the shipping destination name and address",
    "invoice_number": "the invoice number or invoice ID",
    "line_items": "the list of purchased items, products, or services with quantity and price",
    "financial_summary": "the subtotal, tax, and total amount due",
    "notes": "additional notes, remarks, terms, or policies",
}
```

Each of the model's own output keys (plus a short peek at its nested key names, for context) is embedded into the same vector space as the descriptions above, and compared by cosine similarity (a plain matrix multiply of normalized vectors - sims = key_vecs @ canon_vecs.T). A key is renamed to its best-matching canonical concept only if that similarity clears a threshold of 0.42; below that, the key is left exactly as the model wrote it, rather than force-mapping it to a nearest - possibly wrong - concept. This step is deterministic vector math against a separately-running small model, not a second call to the big LLM, and every rename is reported back in field_renames so nothing changes silently.

11. Summary - Who Does What

Step	Owner
Reading the file bytes / dispatching by extension	Python
Native PDF text extraction	Python (PyMuPDF)
Optical character recognition	A separate OCR model (PaddleOCR), not the LLM
Page image rendering for vision	Python (PyMuPDF pixmap -> PNG)
Deciding vision-hybrid vs text-only	Python (a threshold check on page count/char count)
Deciding WHAT JSON keys to use for this document	The LLM, guided by prose rules
Deciding which table rows are real line items	The LLM, guided by prose rules
Resolving OCR-ambiguous characters	The LLM (cross-checking text against the page image)
Extracting the actual values	The LLM
Enforcing a caller-supplied strict schema, if given	vLLM guided decoding (token-level, not the LLM's free choice, not Python post-processing)
Parsing the model's text back into an object	Python (_try_json, regex-based recovery)
Merging N self-consistency samples	Python (_merge_by_vote, pure vote logic)
Finding 'the amount' / 'the vendor' etc. under whatever key name was used	Python (alias-list matching: _first, _find_line_items, _find_totals, _row_amount)
Checking the math (totals, quantity*price, duplicates)	Python (validate_extraction, no model involved)
Fixing a flagged hard issue	The LLM again, one guided pass (_self_correct)

Step	Owner
Removing accidental duplicate values	Python (<code>_dedupe_duplicate_values</code>)
Normalizing field names across documents (optional)	A small embedding model + Python cosine similarity, not the big LLM (<code>canonicalize_fields</code>)

Every function name, line number, constant, and prompt excerpt above was read directly from the live gateway/main.py on 2026-08-21 - none of it is inferred from documentation or API behavior alone.